VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Exercise Report

Báo cáo bài tập E1, E2, E3

Advisor(s): TS. Lê Thành Sách và TS. Nguyễn Quốc Minh
Student(s): Trần Hoài Tâm 2470743

HO CHI MINH CITY, NOVEMBER 2026

Thông tin nộp bài

Học viên: Trần Hoài Tâm

MSHV: 2470743

Link mã nguồn: https://github.com/TamTran9x/2470743-CO5085-Exercise1

Mục lục

1 Dữ liệu và thiết lập chung **6**
1.1 Tập dữ liệu . . . 6
1.2 Chia dữ liệu và tiền xử lý . . . 6
1.3 Vòng lặp huấn luyện . . . 6
1.4 Môi trường . . . 7

2 E1 — Phân loại ảnh bằng Softmax, MLP và CNN **8**
2.1 Kiến trúc . . . 8
2.2 Huấn luyện . . . 8
2.3 Kết quả . . . 9
2.4 Nhận xét . . . 12

3 E2 — Transformer: Multi-Head Self-Attention và tokenization **13**
3.1 MSA tự viết . . . 13
3.2 Bản PyTorch và kiểm chứng tính đúng . . . 14
3.3 Bộ phân loại ảnh . . . 14
3.4 Các cách lấy token . . . 15
3.5 Thiết lập so sánh . . . 15
3.6 Kết quả . . . 16
3.7 Nhận xét . . . 18

4 E3 — LSTM và GRU trên chuỗi ảnh **19**
4.1 Biến ảnh thành chuỗi . . . 19
4.2 Mô hình . . . 20
4.3 Baseline không hồi quy . . . 20
4.4 Kết quả . . . 20
4.5 Khả năng tích lũy thông tin theo thời gian . . . 21
4.6 Nhận xét . . . 22

5 Khai báo sử dụng công cụ AI **23**

A Cấu trúc repository và cách chạy **23**

Danh sách hình vẽ

2.1 Đường cong loss và accuracy của E1 (nét chấm: train, nét liền: validation). 9
2.2 Confusion matrix của CNN trên tập test (số trong ô là số ảnh, màu là tỉ lệ theo hàng). 10
2.3 Tỉ lệ lỗi theo cặp lớp của ba mô hình (đã bỏ đường chéo, cùng thang màu). 11
2.4 Một số ảnh test CNN đoán sai với xác suất dự đoán cao nhất. 11
3.1 Đường cong học của E2. Màu: tokenizer; nét liền: MSA tự viết, nét gạch: PyTorch. 16
3.2 Thời gian forward + backward của một lớp MSA theo số token (trục log–log). 17
3.3 Attention của token CLS ở khối encoder cuối (patch 4×4, MSA tự viết). . 17
4.1 Đường cong học của E3 và hai baseline. Xanh: LSTM, cam: GRU; nét liền: theo hàng, nét gạch: patch. 21
4.2 Test accuracy theo phần trăm số pixel đã thấy (đọc từ trên xuống). 21

Danh sách bảng

2.1 Ba bộ phân loại của E1. Cả ba trả về 10 logits; softmax được áp trong hàm loss. 8
2.2 Kết quả E1 trên Fashion-MNIST. Giá trị cao nhất in đậm. 9
2.3 Accuracy (%) theo từng lớp trên tập test. 10
2.4 Các cặp lớp bị nhầm nhiều nhất (số ảnh test). 11
3.1 Sai lệch giữa MSA tự viết và `nn.MultiheadAttention`. 14
3.2 Các cách tokenize ảnh 28 × 28 trong E2. 15
3.3 Kết quả E2 theo tokenizer và cài đặt MSA. 16
3.4 MSA tự viết so với `nn.MultiheadAttention` trên cùng tokenizer. 16
3.5 Chi phí một lớp MSA theo số token N. 17
4.1 Kết quả E3: LSTM và GRU trên hai cách biểu diễn chuỗi, so với MLP và CNN. 20

Danh sách đoạn mã

1.1 Vòng lặp huấn luyện tự viết (rút gọn từ `common/engine.py`) 6
3.1 Phần forward của MSA tự viết (`E2/msa.py`) 13
A.1 Cấu trúc repository 23

1 Dữ liệu và thiết lập chung

1.1 Tập dữ liệu

Cả ba bài dùng chung tập **Fashion-MNIST**: ảnh xám 28×28 của 10 loại quần áo, giày dép và túi (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot), gồm 60 000 ảnh huấn luyện và 10 000 ảnh kiểm tra, các lớp cân bằng (6 000 ảnh/lớp ở tập huấn luyện).

Fashion-MNIST được chọn thay vì MNIST vì MNIST quá dễ: một bộ phân loại tuyến tính đã đạt khoảng 92%, nên khoảng cách giữa softmax, MLP và CNN rất nhỏ và khó phân tích. Fashion-MNIST có các lớp gần nhau về hình dạng (Shirt, T-shirt/top, Pullover, Coat), tạo ra lỗi có cấu trúc để phân tích. So với CIFAR-10, ảnh 28×28 một kênh giúp các thí nghiệm Transformer và LSTM/GRU chạy nhanh hơn và rất tiện để biểu diễn thành chuỗi 28 hàng là 28 bước thời gian).

1.2 Chia dữ liệu và tiền xử lý

- 60 000 ảnh huấn luyện gốc được hoán vị bằng seed cố định 42: 54 000 ảnh để *train*, 6 000 ảnh để *validation*. Tập *test* gốc 10 000 ảnh chỉ dùng một lần cho kết quả cuối.
- Mọi mô hình ở E1–E3 dùng đúng cùng một split (có test tự động kiểm tra trong `tests/test_all.py`).
- Ảnh được đưa về $[0, 1]$ rồi chuẩn hóa $x \leftarrow (x - \mu)/\sigma$ với $\mu = 0.2858$, $\sigma = 0.3529$ tính *chỉ* trên phần train. Không dùng tăng cường dữ liệu, để so sánh các kiến trúc trong cùng điều kiện.

1.3 Vòng lặp huấn luyện

Vòng lặp được tự viết trong `common/engine.py`, dùng chung cho cả ba bài, đúng trình tự forward → loss → `backward()` → `optimizer.step()`:

Đoạn mã 1.1: Vòng lặp huấn luyện tự viết (rút gọn từ `common/engine.py`)

```
for x, y in loader:
    logits = model(x)                  # forward, (B, 10), not softmaxed
    loss = criterion(logits, y)        # CrossEntropyLoss = log-softmax + NLL
    optimizer.zero_grad(set_to_none=True)
    loss.backward()                    # backward
    if grad_clip > 0:
```

```
        nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
    optimizer.step()                    # update
    scheduler.step()                    # per-step learning rate
```

- **Loss:** `nn.CrossEntropyLoss` nhận logits; softmax nằm sẵn trong loss nên các mô hình không đặt softmax ở cuối `forward`.
- **Optimizer:** AdamW, không áp weight decay cho bias, tham số chuẩn hóa, token CLS và positional embedding.
- **Learning rate:** cập nhật theo từng bước, warmup tuyến tính (nếu có) rồi giảm theo cosine về 0.
- **Chọn mô hình:** sau mỗi epoch đánh giá trên validation; lưu checkpoint có val accuracy cao nhất. Kết quả test dùng checkpoint này, không chọn theo test.
- **Ghi log:** mỗi lần chạy lưu `history.csv` (loss/accuracy train và val, LR, thời gian mỗi epoch), `summary.json`, `best.pt` và logits trên tập test.
- **Tái lập:** cố định seed cho Python, NumPy, PyTorch, thứ tự batch; bật chế độ deterministic của cuDNN.

1.4 Môi trường

Các thí nghiệm chạy trên Tesla T4, PyTorch 2.11.0+cu130. Thời gian mỗi epoch được đo sau khi đồng bộ GPU (`torch.cuda.synchronize`) và lấy trung bình từ epoch thứ 2 để bỏ chi phí khởi động. Cách chạy lại toàn bộ thí nghiệm được ghi trong `README.md` của repository.

2 E1 — Phân loại ảnh bằng Softmax, MLP và CNN

2.1 Kiến trúc

Mô hình	Cấu trúc	Số tham số
Softmax	Flatten (784) → Linear(784, 10)	$784 \cdot 10 + 10 = 7\,850$
MLP	Flatten → Linear(784, 512) → ReLU → Dropout(0.2) → Linear(512, 256) → ReLU → Dropout(0.2) → Linear(256, 10)	535 818
CNN	[Conv3×3(32)–BN–ReLU]×2 → MaxPool → [Conv3×3(64)–BN–ReLU]×2 → MaxPool → Flatten (3136) → Linear(3136, 128) → ReLU → Dropout(0.3) → Linear(128, 10)	468 010

Bảng 2.1: Ba bộ phân loại của E1. Cả ba trả về 10 logits; softmax được áp trong hàm loss.

- **Softmax** là hồi quy logistic đa lớp: $p(y \mid x) = \text{softmax}(Wx + b)$, ranh giới quyết định giữa hai lớp bất kỳ là một siêu phẳng trong không gian 784 chiều.
- **MLP** thêm hai tầng ẩn phi tuyến nhưng vẫn nhận vector flatten, nên không biết hai pixel nào nằm cạnh nhau. Số tham số: $784 \cdot 512 + 512 + 512 \cdot 256 + 256 + 256 \cdot 10 + 10 = 535\,818$.
- **CNN** dùng tích chập 3×3 (tính cục bộ, chia sẻ trọng số) và pooling. Conv đặt `bias=False` vì BatchNorm ngay sau đã có tham số dịch. Phần tích chập chỉ có 65 184 tham số; phần lớn tham số (401 536) nằm ở tầng Linear(3136, 128). Kích thước được chọn để CNN và MLP có số tham số cùng bậc, giúp so sánh công bằng hơn.

2.2 Huấn luyện

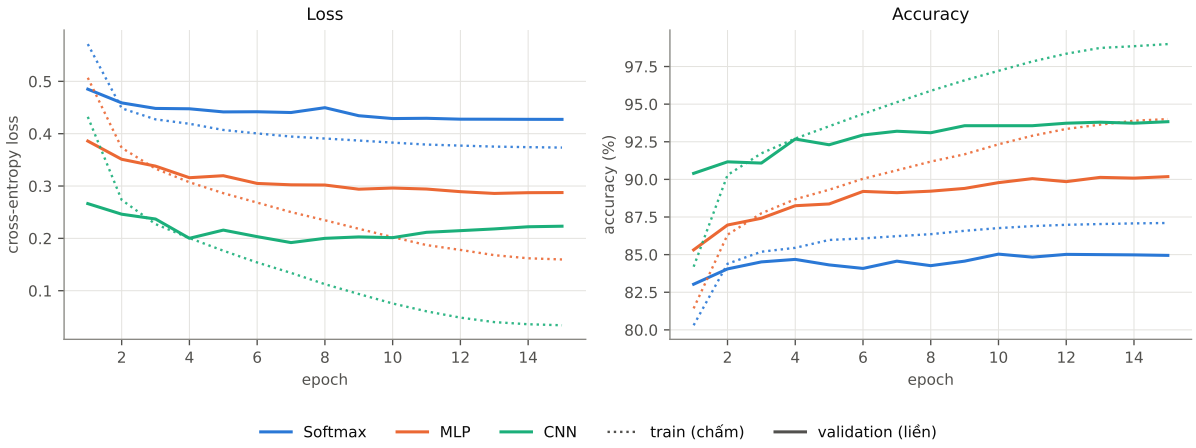
Cả ba mô hình dùng cùng cấu hình: 15 epoch, batch 128, AdamW với LR 10^{-3}, weight decay 10^{-4}, cosine schedule; cùng seed và cùng thứ tự batch.

2.3 Kết quả

Mô hình	Số tham số	Train acc (%)	Val acc tốt nhất (%)	Epoch tốt nhất	Test acc (%)	Test loss	Giây/epoch
Softmax	7 850	87.10	85.03	10	84.51	0.4365	1.0
MLP	535 818	94.01	90.18	15	89.90	0.3035	1.3
CNN	468 010	98.99	93.83	15	**93.51**	0.2290	3.9

Train acc lấy ở epoch cuối (có dropout); test dùng checkpoint có val acc cao nhất.

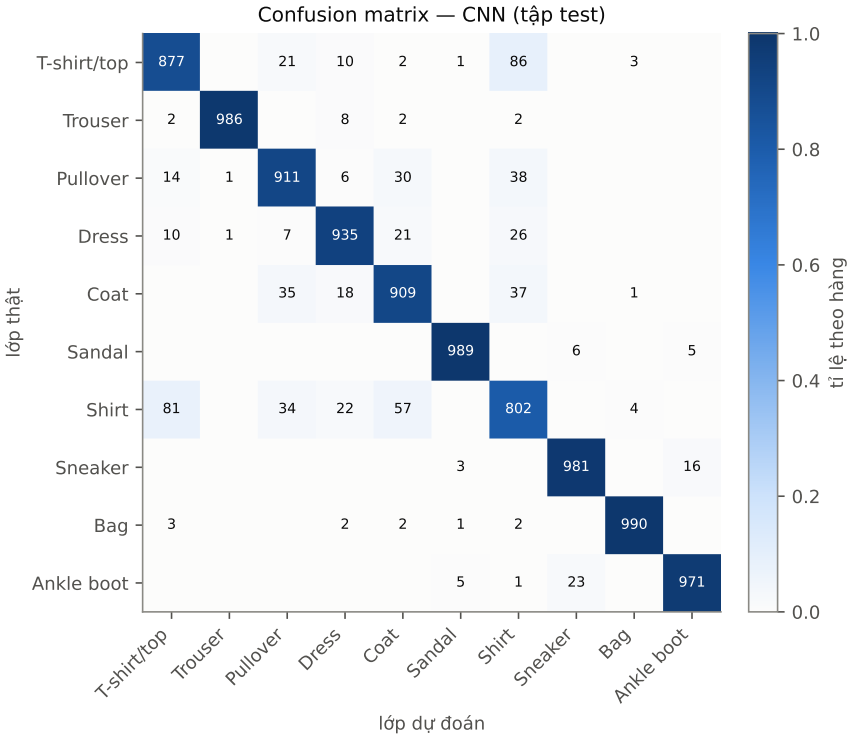
Bảng 2.2: Kết quả E1 trên Fashion-MNIST. Giá trị cao nhất in đậm.



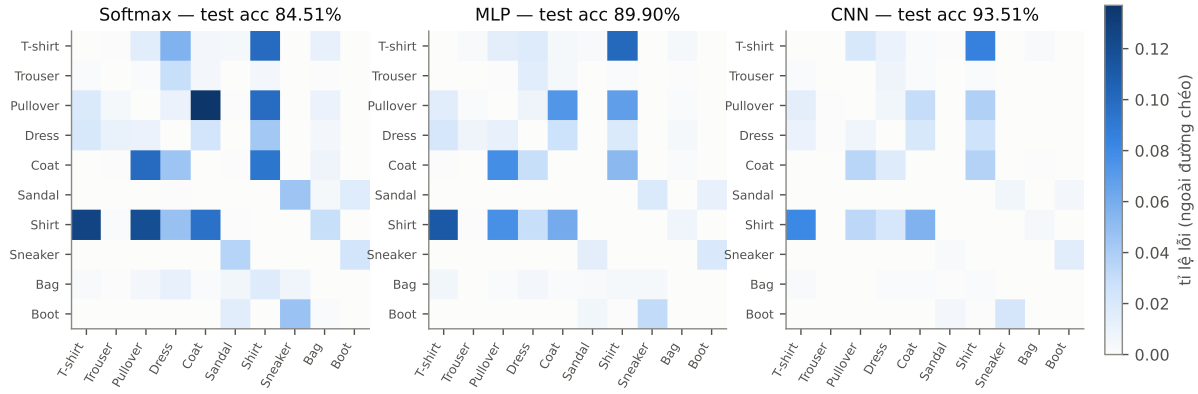
Hình 2.1: Đường cong loss và accuracy của E1 (nét chấm: train, nét liền: validation).

Lớp	Softmax	MLP	CNN
T-shirt/top	80.50	85.30	**87.70**
Trouser	95.60	97.60	**98.60**
Pullover	72.00	83.20	**91.10**
Dress	88.60	90.80	**93.50**
Coat	75.10	83.70	**90.90**
Sandal	93.20	96.90	**98.90**
Shirt	57.90	71.40	**80.20**
Sneaker	94.00	96.60	**98.10**
Bag	94.60	97.40	**99.00**
Ankle boot	93.60	96.10	**97.10**

Bảng 2.3: Accuracy (%) theo từng lớp trên tập test.



Hình 2.2: Confusion matrix của CNN trên tập test (số trong ô là số ảnh, màu là tỉ lệ theo hàng).

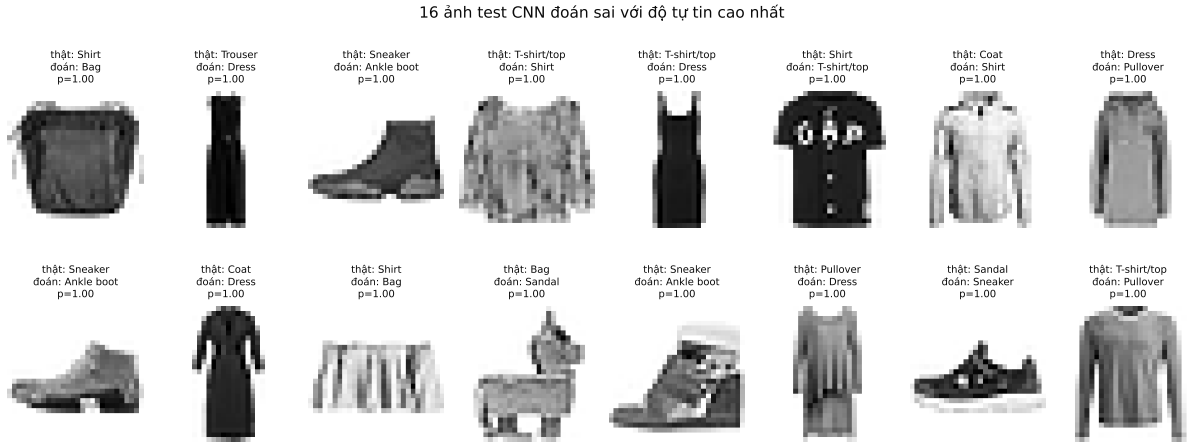


Hình 2.3: Tỉ lệ lỗi theo cặp lớp của ba mô hình (đã bỏ đường chéo, cùng thang màu).

Nhãn thật	Bị đoán là	Softmax	MLP	CNN
T-shirt/top	Shirt	100	101	86
Shirt	T-shirt/top	126	111	81
Shirt	Coat	96	61	57
Pullover	Shirt	99	68	38
Coat	Shirt	93	53	37
Coat	Pullover	100	78	35

Sắp theo số lỗi của CNN trên 10 000 ảnh test.

Bảng 2.4: Các cặp lớp bị nhầm nhiều nhất (số ảnh test).



Hình 2.4: Một số ảnh test CNN đoán sai với xác suất dự đoán cao nhất.

2.4 Nhận xét

- **Thứ hạng rõ ràng:** Softmax 84.51% → MLP 89.90% → CNN 93.51% trên test. Số ảnh sai giảm từ 1 549 xuống 1 010 rồi 649: CNN bớt khoảng một phần ba số lỗi so với MLP.
- **Số tham số không quyết định:** CNN có 468 010 tham số, *ít hơn* MLP (535 818) nhưng chính xác hơn. Phần tích chập chỉ 65 184 tham số, khoảng 86% tham số nằm ở tầng Linear(3136, 128). Lợi thế đến từ inductive bias (tính cục bộ, chia sẻ trọng số, pooling), không đến từ capacity.
- **Đường cong học (Hình 2.1):**
  - Softmax: val loss vẫn giảm đến epoch cuối, train acc 87.10% chỉ cao hơn val 2.07 điểm. Mô hình bị giới hạn bởi capacity (underfit), không overfit.
  - CNN: train loss xuống 0.0340; val loss thấp nhất ở epoch 7 (0.1919) rồi tăng lên 0.2232, trong khi val accuracy vẫn tăng đến epoch 15. Mô hình ngày càng tự tin quá mức với các ảnh nó đoán sai: loss phạt nặng lỗi tự tin, accuracy chỉ đếm argmax.
  - Nếu chọn checkpoint theo val loss thay vì val accuracy, CNN sẽ dừng ở epoch 7 thay vì 15.
- **Theo lớp (Bảng 2.3):** Shirt là lớp khó nhất với cả ba mô hình (57.90% → 71.40% → 80.20%). CNN tăng mạnh nhất so với MLP ở Pullover (+7.9 điểm), Shirt (+8.8) và Coat (+7.2). Trouser, Sandal, Sneaker, Bag, Ankle boot đã trên 93% ngay với softmax vì hình dạng tổng thể đủ để phân biệt.
- **Lỗi có cấu trúc (Bảng 2.4):** cặp nhầm nhiều nhất là T-shirt/top → Shirt (86 ảnh với CNN). Tỉ lệ lỗi nằm trong nhóm áo thân trên (T-shirt/top, Pullover, Coat, Shirt) tăng dần 58.7% → 65.0% → 67.0%: mô hình càng tốt thì lỗi còn lại càng dồn vào nhóm vốn mơ hồ ở độ phân giải 28×28.
- **Lỗi chồng lấp:** 70.1% số ảnh CNN đoán sai cũng bị MLP đoán sai, và 389 ảnh cả ba mô hình đều sai. Phần lớn lỗi nằm ở ảnh khó thật sự chứ không phải ở điểm yếu riêng của một kiến trúc.
- **Lỗi tự tin (Hình 2.4):** CNN đoán sai với xác suất gần 1.00. Nhiều ảnh nhìn bằng mắt giống lớp được dự đoán hơn lớp được gán (ví dụ ảnh nhãn Shirt trông như túi, nhãn Trouser trông như váy), cho thấy tập dữ liệu có nhiễu nhãn và xác suất softmax không phản ánh đúng độ chắc chắn.

- **Chi phí:** trên T4, CNN mất 3.9 s/epoch so với 1.3 s của MLP và suy luận 10 000 ảnh mất 0.384 s so với 0.071 s, dù ít tham số hơn: mỗi trọng số tích chập được dùng lại ở mọi vị trí nên số phép tính trên mỗi tham số lớn hơn nhiều.

3 E2 — Transformer: Multi-Head Self-Attention và tokenization

3.1 MSA tự viết

Với chuỗi token $X \in \mathbb{R}^{N\times D}$, h head và $d_k = D/h$:

$$
\begin{aligned}
Q &= XW_Q + b_Q, \quad K = XW_K + b_K, \quad V = XW_V + b_V \\
A_i &= \mathrm{softmax}\!\left(\frac{Q_i K_i^{\top}}{\sqrt{d_k}}\right) \in \mathbb{R}^{N\times N}, \qquad \mathrm{head}_i = A_i V_i, \quad i = 1,\dots,h \\
\mathrm{MSA}(X) &= \mathrm{concat}(\mathrm{head}_1,\dots,\mathrm{head}_h)\, W_O + b_O
\end{aligned}
$$

trong đó Q_i, K_i, V_i là khối cột thứ i (độ rộng d_k) của Q, K, V. Bản tự viết (`E2/msa.py`, lớp `MultiHeadSelfAttentionScratch`) chỉ dùng `nn.Linear`, `torch.matmul`, `torch.softmax` và các phép đổi shape:

Đoạn mã 3.1: Phần forward của MSA tự viết (`E2/msa.py`)

```
q = self._split_heads(self.q_proj(x))                    # (B, h, N, d_k)
k = self._split_heads(self.k_proj(x))
v = self._split_heads(self.v_proj(x))
scores = torch.matmul(q, k.transpose(-2, -1)) * self.scale  # (B, h, N, N)
attn = torch.softmax(scores, dim=-1)
out = torch.matmul(self.attn_drop(attn), v)              # (B, h, N, d_k)
out = out.transpose(1, 2).reshape(B, N, D)               # concat heads
return self.out_proj(out)                                # (B, N, D)
```

`_split_heads` đổi $(B, N, D) \to (B, N, h, d_k) \to (B, h, N, d_k)$ để mọi head được tính song song bằng một phép nhân ma trận theo lô. Số tham số của một lớp MSA là $4(D^2+D)$. Trọng số được khởi tạo giống `nn.MultiheadAttention`: Xavier-uniform trên ma trận ghép $[W_Q; W_K; W_V]$ kích thước $3D \times D$, bias bằng 0.

3.2 Bản PyTorch và kiểm chứng tính đúng

Bản đối chiếu `MultiHeadSelfAttentionTorch` bọc `nn.MultiheadAttention` (với `batch_first=True`). Để kiểm chứng, trọng số của `nn.MultiheadAttention` được chép sang bản tự viết (`in_proj_weight` là $[W_Q; W_K; W_V]$ xếp chồng, cộng `out_proj`), rồi cùng một đầu vào được đưa qua hai bản để so sánh đầu ra, ma trận attention của từng head và gradient theo đầu vào (`E2/check_msa.py`, cùng các test tự động trong `tests/test_all.py`).

D	h	N	kiểu số	lệch output	lệch attention	lệch gradient	kết quả
32	4	10	float64	2.8e-16	1.7e-16	4.4e-16	đạt
96	4	50	float64	2.2e-16	1.7e-16	3.9e-16	đạt
96	8	29	float64	2.8e-16	1.7e-16	2.8e-16	đạt
64	1	17	float64	0.0e+00	0.0e+00	5.6e-17	đạt
32	4	10	float32	8.9e-08	8.9e-08	1.3e-07	đạt
96	4	50	float32	1.3e-07	1.3e-07	1.5e-07	đạt
96	8	29	float32	1.3e-07	1.2e-07	1.5e-07	đạt
64	1	17	float32	1.3e-07	1.0e-07	2.5e-07	đạt

Sai lệch tuyệt đối lớn nhất giữa bản tự viết và nn.MultiheadAttention khi dùng chung trọng số (chạy trên cuda).

Bảng 3.1: Sai lệch giữa MSA tự viết và `nn.MultiheadAttention`.

Sai lệch lớn nhất ở đầu ra là 3e-16 với float64 và 1e-07 với float32, tức chỉ ở mức sai số làm tròn số thực: hai cài đặt tính cùng một hàm.

3.3 Bộ phân loại ảnh

Ảnh → tokenizer → thêm token CLS → cộng positional embedding học được → 4 khối encoder → LayerNorm → Linear trên token CLS. Mỗi khối encoder dùng dạng pre-norm:

$$X \leftarrow X + \mathrm{Dropout}\big(\mathrm{MSA}(\mathrm{LN}(X))\big), \qquad X \leftarrow X + \mathrm{FFN}(\mathrm{LN}(X)),$$

với FFN là Linear(D, rD)–GELU–Linear(rD, D). Cấu hình: $D = 96$, $h = 4$ head, $r = 2.0$, dropout 0.1. Khối encoder được tự lắp (không dùng `nn.TransformerEncoderLayer`) để bản tự viết và bản PyTorch chỉ khác nhau đúng ở phần MSA.

3.4 Các cách lấy token

Tokenizer	Mỗi token là	N (chưa gồm CLS)
Patch 4×4	một ô 4×4 không chồng nhau, chiếu tuyến tính 16 $\to D$	49
Patch 7×7	một ô 7×7 không chồng nhau, chiếu tuyến tính 49 $\to D$	16
Theo hàng	một hàng 28 pixel, chiếu tuyến tính $28 \to D$	28
CNN-stem	một vị trí trên feature map 7×7 của CNN 2 tầng (32, 64 kênh) + conv 1×1 về D kênh	49

Bảng 3.2: Các cách tokenize ảnh 28×28 trong E2.

Patch embedding được cài bằng `nn.Conv2d` với `kernel_size = stride = p`. Phép này tương đương hoàn toàn với cắt ảnh thành các ô $p \times p$, flatten từng ô rồi nhân với cùng một ma trận chiếu (được kiểm tra trong test `test_patch_conv_equals_patchify_plus_linear`). Token theo hàng có ý nghĩa gần với cách E3 đưa ảnh vào LSTM/GRU, nên hai bài so sánh được với nhau.

3.5 Thiết lập so sánh

- **Tự viết vs. PyTorch (cùng tokenizer):** hai lần chạy xuất phát từ *cùng* trọng số khởi tạo (dựng model PyTorch với seed cố định rồi chép sang bản tự viết), cùng thứ tự batch và dropout attention bằng 0. Khác biệt còn lại chỉ đến từ cách tính MSA (kernel `scaled_dot_product_attention` của PyTorch so với các phép toán tường minh).
- **Giữa các tokenizer:** cùng encoder, cùng siêu tham số, 20 epoch, AdamW (LR 10^{-3}, weight decay 0.05), warmup 1 epoch rồi cosine.

3.6 Kết quả

Tokenizer	MSA	Số token	Số tham số	Train acc	Val acc	Test acc	Giây/epoch	Suy luận test (s)
Patch 4×4	tự viết	50	306 826	93.18	89.92	89.61	6.1	0.40
Patch 4×4	PyTorch	50	306 826	93.46	90.02	89.67	6.7	0.32
Patch 7×7	tự viết	17	306 826	93.68	89.93	89.45	5.6	0.17
Theo hàng	tự viết	29	305 962	95.02	90.63	90.09	5.5	0.28
Theo hàng	PyTorch	29	305 962	95.00	90.68	89.77	5.0	0.21
CNN-stem	tự viết	50	330 346	96.43	92.63	**92.07**	7.5	0.47

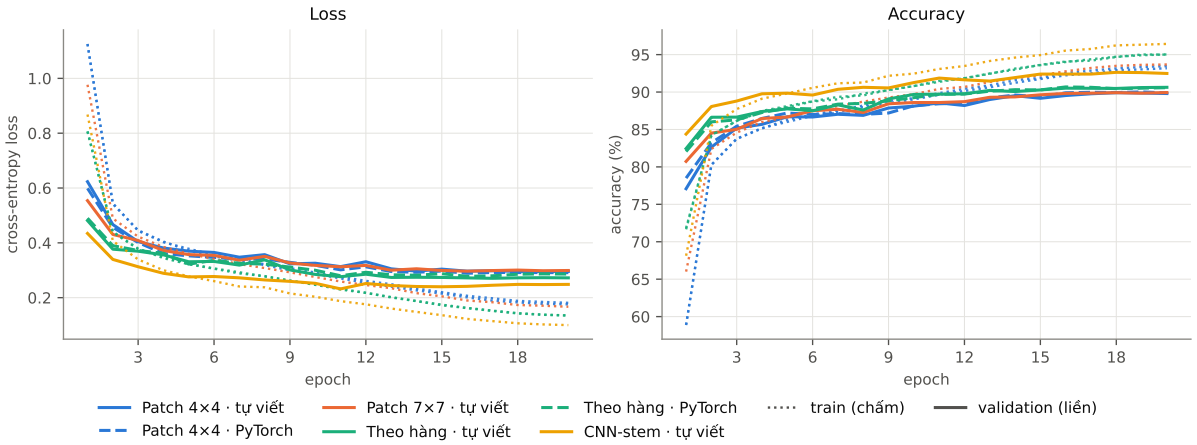
Accuracy tính theo %. Số token đã gồm token CLS. Suy luận: toàn bộ 10 000 ảnh test.

Bảng 3.3: Kết quả E2 theo tokenizer và cài đặt MSA.

Tokenizer	Test acc tự viết	Test acc PyTorch	Chênh (điểm %)	Giây/epoch tự viết	Giây/epoch PyTorch	Tỉ lệ thời gian
Patch 4×4	89.61	89.67	-0.06	6.1	6.7	0.91
Theo hàng	90.09	89.77	+0.32	5.5	5.0	1.09

Hai bản xuất phát từ cùng trọng số khởi tạo và cùng thứ tự batch.

Bảng 3.4: MSA tự viết so với `nn.MultiheadAttention` trên cùng tokenizer.

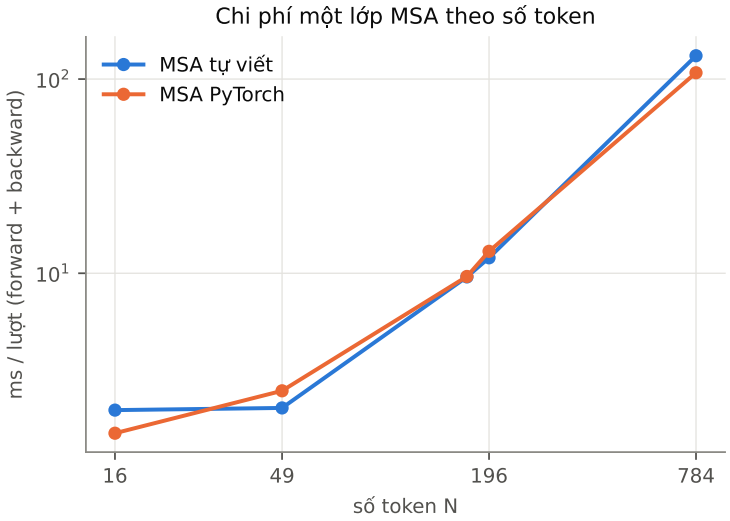


Hình 3.1: Đường cong học của E2. Màu: tokenizer; nét liền: MSA tự viết, nét gạch: PyTorch.

N	tự viết (ms)	PyTorch (ms)	tỉ lệ	bộ nhớ tự viết (MB)	bộ nhớ PyTorch (MB)	ma trận A (MB)
16	1.97	1.50	1.32	26	35	0
49	2.03	2.48	0.82	50	54	5
169	9.58	9.63	0.99	281	138	56
196	12.01	12.97	0.93	368	162	75
784	131.87	107.76	1.22	5004	567	1200

Một lớp MSA, forward + backward, batch 128, $D = 96$, $h = 4$, trung bình 20 lượt trên cuda.

Bảng 3.5: Chi phí một lớp MSA theo số token N.



Hình 3.2: Thời gian forward + backward của một lớp MSA theo số token (trục log–log).



Hình 3.3: Attention của token CLS tới 49 patch ở khối encoder cuối (trung bình 4 head), mô hình patch 4×4 dùng MSA tự viết. Mỗi cột là ảnh test đầu tiên của một lớp; hàng dưới phủ bản đồ attention lên ảnh (sáng = cao). Hình sinh từ notebook `E2/E2.ipynb`.

3.7 Nhận xét

- **MSA tự viết đúng:** sai lệch lớn nhất so với `nn.MultiheadAttention` là 3e-16 (float64) và 1e-07 (float32) ở cả đầu ra, ma trận attention lẫn gradient; số tham số bằng nhau ($4(D^2 + D)$).
- **Huấn luyện từ cùng khởi tạo (Bảng 3.4):**
  - Test accuracy chênh 0.06 điểm (patch 4×4) và 0.32 điểm (theo hàng).
  - Hai đường cong gần như trùng nhau (Hình 3.1). Val accuracy lệch lớn nhất 1.42 điểm ở epoch đầu, sau đó dưới 1 điểm.
  - Hai bản tính cùng một hàm. Khác biệt chỉ do sai số làm tròn (thứ tự phép tính khác nhau giữa các kernel) tích lũy qua khoảng 8 400 bước cập nhật, nhỏ hơn mức dao động giữa các epoch.
- **Tốc độ huấn luyện không có bên thắng rõ:** tỉ lệ thời gian/epoch tự viết so với PyTorch là 0.91 (patch 4×4) và 1.09 (theo hàng). Với $N \leq 50$ token, ma trận attention rất nhỏ nên thời gian chủ yếu nằm ở các tầng Linear, FFN, LayerNorm và chi phí gọi kernel.Chênh lệch chỉ khoảng 10% và ngược chiều nhau giữa hai tokenizer, nên không kết luận được bên nào nhanh hơn.
- **Suy luận thì PyTorch nhanh hơn:** 1.24 lần (patch 4×4) và 1.33 lần (theo hàng). Ở chế độ `eval()` không tính gradient, `nn.MultiheadAttention` chuyển sang "fast path", gộp các bước của MSA vào một hàm C++ tối ưu.
- **Khác biệt thật nằm ở bộ nhớ khi N lớn (Bảng 3.5):**
  - Với $N = 784$ (mỗi pixel một token), bản tự viết dùng 5 004 MB, PyTorch chỉ 567 MB (ít hơn 8.8lần); thời gian 131.9 so với 107.8 ms.
  - Riêng ma trận A đã chiếm 1 200 MB và bản tự viết phải lưu nó cho backward. Kernel `scaled_dot_product_attention` của PyTorch không tạo ra toàn bộ ma trận $N \times N$ mà tính lại khi backward.
- **Chi phí theo N (Hình 3.2):** từ 49 lên 784 token (N tăng 16 lần) thời gian tăng khoảng 65 lần.
  - Ở N nhỏ thời gian gần như không đổi (2.0 ms với 16 token, 2.0 ms với 49 token) vì GPU chưa được dùng hết và chi phí gọi kernel chiếm phần lớn.
  - Ở N lớn phần N^2 của attention chiếm ưu thế: từ 196 lên 784 token (N tăng 4 lần) thời gian tăng khoảng 11 lần.

- **So sánh tokenizer:** CNN-stem 92.07% > theo hàng 90.09% > patch 4×4 89.61% ≈ patch 7×7 89.45%.
  - Patch 7×7 chỉ có 16 token mà gần bằng patch 4×4 (49 token), trong khi nhanh hơn (5.6 so với 6.1 s/epoch; suy luận 0.172 so với 0.402 s). Tăng gấp ba số token không đem lại lợi ích ở bài này.
  - Token CNN-stem nhìn vùng 10×10 pixel (receptive field sau hai conv 3×3 và haimaxpool) và các vùng chồng lên nhau, đã qua phi tuyến, nên mang đặc trưng cục bộ tốt hơn một ô 4×4 chỉ chiếu tuyến tính.
- **Token CLS nhìn vào đâu (Hình 3.3):** ở khối cuối, attention của token CLS khá thưa, mỗi ảnh chỉ dồn vào vài patch. Một số vị trí dễ hiểu: khe giữa hai ống quần (Trouser), mép dưới của áo (Pullover, Shirt), góc túi (Bag). Với Dress, attention trải lên phần nền hai bên váy, tức mô hình dựa vào đường viền tổng thể. Tuy vậy, đến khối thứ 4 mỗi token đã trộn thông tin từ các token khác qua các khối trước, nên bản đồ này chỉ mang tính gợi ý, không giải thích đầy đủ quyết định của mô hình.
- **Overfit:** CNN-stem có train 96.43% so với val 92.63%;val loss thấp nhất ở epoch 11 rồi tăng lại. Các bản patch và theo hàng có val loss thấp nhất ở epoch 16–19, nên có thể còn tăng nhẹ nếu huấn luyện lâu hơn.
- **So với E1:** mọi Transformer (khoảng 306–330 nghìn tham số) đều thua CNN của E1 (93.51%). Các bản chỉ dùng patch ngang mức MLP (89.90%). Với 54 000 ảnh nhỏ và không tăng cường dữ liệu, Transformer thiếu inductive bias không gian; ghép CNN stem vào đã thu hẹp phần lớn khoảng cách.

4 E3 — LSTM và GRU trên chuỗi ảnh

4.1 Biến ảnh thành chuỗi

Ảnh $(1, 28, 28)$ được biến thành chuỗi (T, F) theo hai cách (`E3/models.py`, hàm `to_sequence`):

- **Theo hàng:** bước thời gian t là hàng thứ t của ảnh, đọc từ trên xuống: $T = 28$, $F = 28$.
- **Theo patch 7×7:** ảnh được chia thành lưới 4×4 patch, duyệt theo thứ tự hàng (trái sang phải, trên xuống dưới); mỗi patch flatten thành 49 giá trị: $T = 16$, $F = 49$.

Hai cách này đánh đổi độ dài chuỗi và lượng thông tin mỗi bước: chuỗi theo hàng dài hơn nhưng mỗi bước chỉ thấy một dải mỏng; chuỗi patch ngắn hơn nhưng phá vỡ tính liên tục theo chiều ngang giữa các hàng patch.

4.2 Mô hình

Chuỗi đi qua `nn.LSTM` hoặc `nn.GRU` (2 tầng, hidden 128, dropout 0.2 giữa các tầng, `batch_first=True`); hidden state của bước cuối ở tầng trên cùng đi qua một tầng Linear ra 10 logits. Với kích thước đầu vào F, hidden H và g cổng ($g = 4$ với LSTM: input, forget, cell, output; $g = 3$ với GRU: reset, update, new), số tham số tầng thứ nhất là

$$g\,(HF + H^2 + 2H),$$

nên với cùng H GRU có đúng 3/4 số tham số hồi quy của LSTM. Huấn luyện: 15 epoch, AdamW (LR 10^{-3}, weight decay 10^{-4}), cosine, cắt gradient theo chuẩn 1.0.

4.3 Baseline không hồi quy

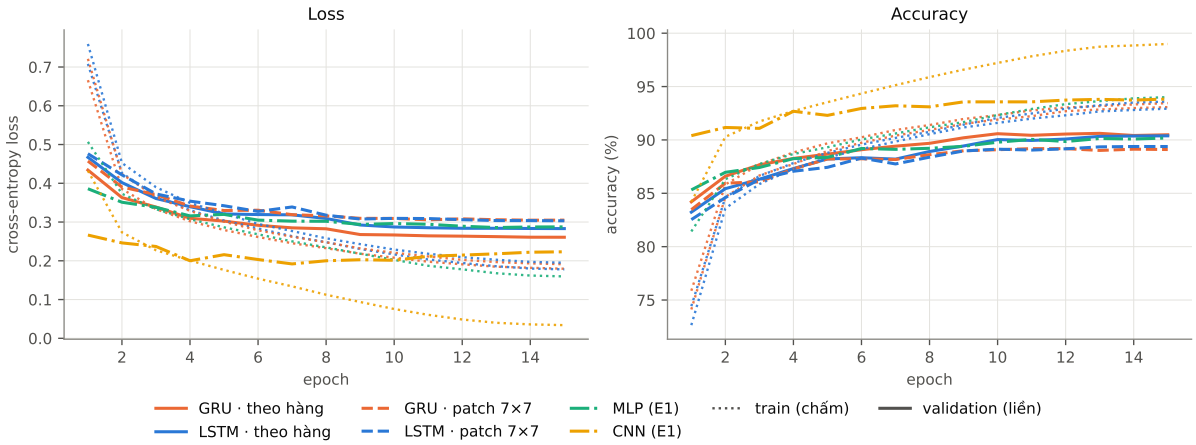
MLP và CNN lấy lại từ E1 (cùng split, cùng seed, checkpoint tốt nhất theo validation).

4.4 Kết quả

Mô hình	Đầu vào	$T \times F$	Số tham số	Train acc	Val acc	Test acc	Giây/epoch
GRU	chuỗi theo hàng	28 × 28	161 034	93.45	90.62	90.60	1.8
LSTM	chuỗi theo hàng	28 × 28	214 282	93.59	90.38	89.87	2.2
GRU	chuỗi patch 7×7	16 × 49	169 098	93.07	89.18	89.15	1.8
LSTM	chuỗi patch 7×7	16 × 49	225 034	92.93	89.38	88.71	1.9
MLP (E1)	không hồi quy	vector 784	535 818	94.01	90.18	89.90	1.3
CNN (E1)	không hồi quy	ảnh 28 × 28	468 010	98.99	93.83	**93.51**	3.9

Accuracy tính theo %. MLP và CNN lấy từ E1 (cùng split, cùng seed).

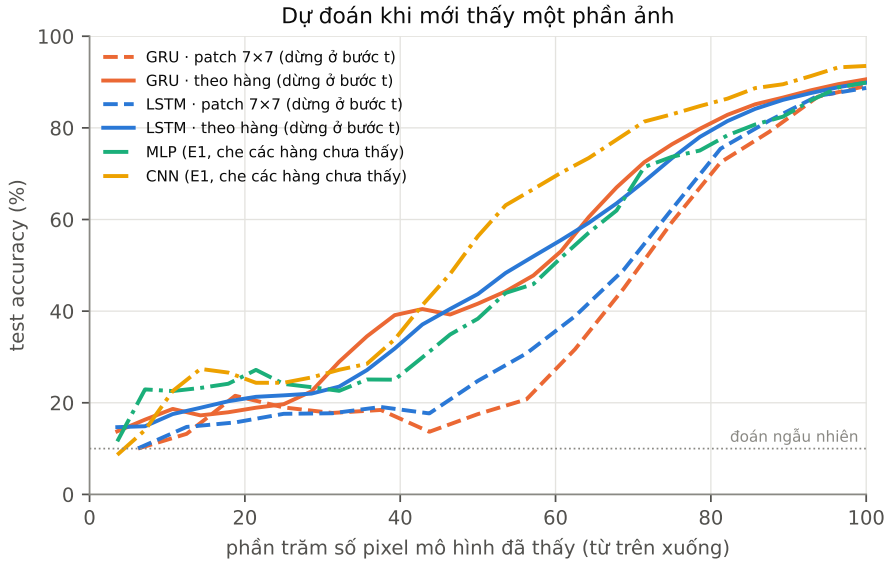
Bảng 4.1: Kết quả E3: LSTM và GRU trên hai cách biểu diễn chuỗi, so với MLP và CNN.



Hình 4.1: Đường cong học của E3 và hai baseline. Xanh: LSTM, cam: GRU; nét liền: theo hàng, nét gạch: patch.

4.5 Khả năng tích lũy thông tin theo thời gian

Vì LSTM/GRU một chiều, output tại bước t chỉ phụ thuộc t phần đầu của chuỗi. Đưa output đó qua tầng phân loại cho biết mô hình dự đoán thế nào khi mới "đọc" được một phần ảnh, không cần huấn luyện lại. Với MLP và CNN, ảnh được giữ k hàng đầu và các hàng còn lại thay bằng pixel đen. Không mô hình nào được huấn luyện với ảnh bị che, nên đây là phép thử như nhau cho cả hai loại mô hình (`E3/temporal.py`).



Hình 4.2: Test accuracy theo phần trăm số pixel đã thấy (đọc từ trên xuống).

4.6 Nhận xét

- **GRU tốt hơn LSTM ở cả hai cách biểu diễn:**
  - Theo hàng: 90.60% so với 89.87%.
  - Patch 7×7: 89.15% so với 88.71%.
  - GRU chỉ dùng khoảng 3/4 tham số hồi quy (161 034 so với 214 282). Một cách giải thích: với chuỗi ngắn (16–28 bước), cell state riêng của LSTM chưa phát huy tác dụng, trong khi GRU ít cổng hơn nên dễ tối ưu hơn trong 15 epoch.
  - Thời gian/epoch gần như nhau (1.8 so với 2.2 s) vì cả hai chạy bằng kernel cuDNN.
- **Theo hàng tốt hơn theo patch** với cả hai loại cell. Chuỗi patch ngắn hơn (16 bước) nhưng thứ tự quét theo hàng làm hai patch liền nhau theo chiều dọc cách nhau 4 bước. Patch cuối mỗi hàng patch (mép phải) lại đứng ngay trước patch đầu hàng sau (mép trái), nên thứ tự thời gian không phản ánh đúng vị trí không gian.
- **So với baseline (Bảng 4.1):**
  - GRU theo hàng (90.60%) vượt MLP (89.90%) với chưa tới một phần ba số tham số. LSTM theo hàng ngang MLP; hai bản theo patch thấp hơn MLP.
  - Mọi RNN đều kém CNN (93.51%) khoảng 3–5 điểm: RNN chỉ khai thác thứ tự theo một chiều, còn CNN khai thác tính cục bộ theo cả hai chiều.
  - Transformer token theo hàng ở E2 đạt 90.09%, xấp xỉ GRU theo hàng: với cùng cách coi mỗi hàng là một phần tử chuỗi, hồi quy và attention cho kết quả tương đương ở bài này.
- **Chưa overfit:** val loss của các RNN vẫn thấp nhất ở những epoch cuối (LSTM theo hàng: epoch 15); khoảng cách train–val khoảng 3–4 điểm. Nhiều khả năng còn cải thiện nếu huấn luyện lâu hơn, khác với CNN đã bắt đầu tăng val loss.
- **Thí nghiệm nhìn một phần ảnh (Hình 4.2) cho kết quả trái với dự đoán ban đầu:**
  - Khi mới thấy 50% số pixel (14 hàng trên), LSTM/GRU theo hàng đạt 43.7% và 41.6%, MLP bị che 38.3%, còn CNN bị che đạt 56.3%. Ở 75%: 73.4%, 76.4%, 73.7% và 83.0%. RNN *không* dự đoán sớm tốt hơn CNN.
  - Accuracy của RNN tăng dần theo số hàng đã đọc và tăng nhanh nhất khi đọc phần giữa ảnh (50–75%). Điều này cho thấy mô hình thực sự tích lũy thông



tin qua hidden state và quyết định dựa trên toàn bộ chuỗi. Nhưng loss chỉ tính ở bước cuối, nên hidden state giữa chừng không được huấn luyện để phân loại.

  - Cùng thấy đúng 14 hàng trên (8 patch đầu), mô hình theo patch chỉ đạt 24.8% (LSTM) và 17.5% (GRU), thấp hơn hẳn bản theo hàng. GRU patch còn giảm từ 19.0% xuống 17.5% khi thấy thêm ảnh. Cách biểu diễn ảnh hưởng mạnh đến việc trạng thái trung gian "đọc" được hay không.
  - Hướng cải thiện nếu muốn mô hình dự đoán sớm: tính loss ở mọi bước, hoặc dùng RNN hai chiều nếu không cần dự đoán sớm.

- **Chi phí:** RNN chỉ mất khoảng 1.8–2.2 s/epoch, nhanh hơn CNN (3.9 s) nhờ chuỗi ngắn và kernel cuDNN. Tuy vậy các bước thời gian phải tính tuần tự nên khó song song hóa khi chuỗi dài.

5 Khai báo sử dụng công cụ AI

Theo quy định của môn học (được phép dùng AI nhưng phải hiểu và bảo vệ được code cùng số liệu), học viên đã sử dụng Claude (Anthropic), ChatGPT (OpenAI) để hỗ trợ tìm hiểu, giải thích và viết mã nguồn file notebook cho các bài tập E1 ,E2 và E3. Học viên cũng tự chạy toàn bộ thí nghiệm trên Google Colab, (GPU Tesla T4), đối chiếu từng nhận xét với số liệu trong `results/`, kiểm tra tính đúng qua các test tự động (`tests/test_all.py`, `E2/check_msa.py`), và chịu trách nhiệm về toàn bộ mã nguồn và số liệu đã nộp.

A Cấu trúc repository và cách chạy

Đoạn mã A.1: Cấu trúc repository

```
common/   data.py   engine.py   utils.py      # split, training loop, utilities
E1/       models.py train.py E1.ipynb
E2/       msa.py tokenizers.py model.py train.py check_msa.py bench_msa.py E2.
    ipynb
E3/       models.py train.py temporal.py E3.ipynb
tests/    test_all.py
scripts/  make_report_assets.py build_notebooks.py
results/  E1/ E2/ E3/                              # history.csv, summary.json,
     test logits
```

```
reports/  exercises.tex  exercises.pdf  generated/
run_all.ipynb   run_all.sh   index.html (GitHub Pages)
```

Chạy toàn bộ: `bash run_all.sh` (hoặc notebook `run_all.ipynb` trên Google Colab), sau đó `cd reports && xelatex exercises.tex` (hai lần). Mỗi notebook `Ei/Ei.ipynb` tự chứa toàn bộ code của bài đó và chạy được độc lập trên Colab.